ENGINEERING ARCHITECTURE AUDIT

Official Technical Report

Food Court Management Application

Comprehensive Full-Stack Codebase Review: Assets, Dependencies, Security & Performance

Stack: FastAPI (Python 3.12) + Next.js 16 (React 19) + PostgreSQL

Date: October 2026

Environment: Local / Production Ready

9.5 / 10	9.8 / 10	7.2 / 10	4.5 / 10
BUSINESS LOGIC	BUILD & RUNTIME	ARCHITECTURE	SECURITY (ACTION)

📊 SYSTEM HEALTH & MODULE STATUS

MODULE / COMPONENT	TECHNOLOGY	HEALTH STATUS	AUDIT ASSESSMENT
Unified Expense Ledger & Khata	FastAPI + Next.js	EXCELLENT	Clean running balance, vendor accounts, 1-click & custom partial settlement logic.
Kitchen Duty & Wages	FastAPI + Next.js	EXCELLENT	Accurate shift tracking, FIFO wage distribution, and settlement calculations.
Events & Customer Billing	FastAPI + Next.js	EXCELLENT	Print-ready invoices with official SVG Food Court Seal; no bank detail clutter.
Excel & Google Drive Backup	XLSX + Google OAuth	EXCELLENT	Multi-tab formatted spreadsheets covering all modules with cloud sync.
Authentication & Authorization	FastAPI + Cookies	NEEDS SECURITY	Plaintext passwords; role header `X-User-Role` needs cryptographic JWT verification.
Database Migrations	Alembic + SQLAlchemy	OUT OF SYNC	Tables rely on dynamic `create_all()`; Alembic migration files stopped updating Oct 3.
Code Organization	FastAPI / Next.js	MONOLITHIC	`main.py` has 88 routes (3.5k lines); `expenses/page.tsx` has 4.3k lines.

🗑️ WHAT IS NOT NEEDED (DEAD CODE, BLOAT & CLEANUP ITEMS)

1. Phantom 0-Byte File: `backend/app/services`

IMMEDIATE DELETION

A 0-byte ghost file created on Sep 29. It is neither a directory nor a valid Python script, yet sits directly inside the core backend application directory.

Action: Delete `backend/app/services` to prevent confusion.

2. Orphaned Route: `frontend/app/events/edit/[ id ]/`

IMMEDIATE DELETION

This folder contains spaces in its name (`[ id ]`) and exports an outdated component. No route, navigation drawer link, or button in the entire project links to it. Event editing is already handled inside `events/[id]/page.tsx` .

Action: Delete the entire folder `frontend/app/events/edit/` .

3. Duplicate Pydantic Schema in `backend/app/schemas.py`

CODE HYGIENE

`class CreateEventRequest(BaseModel)` is defined at line 104 and completely overwritten 50 lines later at line 154. The initial definition is completely unreachable dead code.

Action: Remove lines 104–134 in `backend/app/schemas.py` .

4. Deprecated ESLint Option in `frontend/next.config.ts`

BUILD WARNING

Next.js 16 deprecated `eslint: { ignoreDuringBuilds: true }` . Keeping it causes TypeScript compilation error TS2353 and invalid configuration warnings during production build.

Action: Remove the `eslint` property from `next.config.ts` .

5. Invalid Docker Compose Tags & Volume Path (`docker-compose.yml`)

DEPLOYMENT FIX

Line 3 specifies `image: postgres:18-alpine` (Postgres 18 does not exist yet). The volume mounts `/var/lib/postgresql` instead of `/var/lib/postgresql/data` .

Action: Change image to `postgres:17-alpine` and mount `postgres_data:/var/lib/postgresql/data` .

🔒 WHAT IS CRITICALLY NEEDED (SECURITY, DATABASE & BACKUPS)

1. Password Hashing (`bcrypt` / `passlib`)

HIGH PRIORITY SECURITY

Vulnerability: Passwords (`admin123` , `owner123` , `manager123`) are currently stored in plain text in the database. During login, the backend evaluates `if user.password != password` .
Consequence: Anyone with access to a database backup, dump file, or read permissions can see all passwords instantly.

Remedy: Install `passlib[bcrypt]` , hash passwords before saving, and verify using `pwd_context.verify(password, user.password_hash)` .

2. Cryptographic JWT Authentication & RBAC

HIGH PRIORITY SECURITY

Vulnerability: The backend role-based access middleware trusts the client-provided header `X-User-Role` without verifying a server-signed secret key.
Consequence: Any external client, Postman script, or curl command can send `X-User-Role: admin` and bypass all manager/owner restrictions.

Remedy: Issue signed JWT tokens on login signed with `SECRET_KEY` . Verify the token on every protected API call.

3. Remove Passwords from Public Endpoints (`GET /auth/roles`)

INFORMATION LEAK

The public endpoint `/auth/roles` explicitly returns `{"default_password": "admin123", ...}` in its JSON response.

Remedy: Remove `default_password` from the endpoint response payload.

4. Complete Alembic Migration Synchronization

DATABASE INTEGRITY

Alembic migration files stopped at version `d19624156c35` on Oct 3. Newer tables (`KitchenStaff` , `KitchenDuty` , `ExpenseSettlement` , `SubTeam`) were created via dynamic fallback. Deploying on a new database with Alembic alone will fail unless migrations are brought up to date.

Remedy: Run `alembic revision --autogenerate -m "sync_full_schema"` to capture all current models.

5. Automated Permanent Drive Backup Script

DATA PROTECTION

While the Google Drive Excel exporter works well for spreadsheets, the actual PostgreSQL database needs daily automated binary dumps (`pg_dump`) stored directly onto your permanent drive or external storage.

Remedy: Create a lightweight Windows `backup_database.bat` scheduled via Windows Task Scheduler to back up to your permanent drive automatically.

🏗️ ARCHITECTURE & MAINTAINABILITY ROADMAP

Modularize FastAPI Monolith (`backend/app/main.py`)

MAINTAINABILITY

Currently, `main.py` contains 88 endpoints across 3,532 lines. Splitting into FastAPI `APIRouter` modules will drastically simplify debugging, testing, and team development:

TARGET MODULE	PROPOSED ROUTE FILE	ENDPOINTS INCLUDED
Authentication & Roles	`app/routers/auth.py`	Login, user sessions, role checks, password resets
Expenses & Supplier Khata	`app/routers/expenses.py`	Ledger, settlements, vendor balances, partial payments
Kitchen Management	`app/routers/kitchen.py`	Staff directory, shift duty tracking, wage calculation
Events & Billing	`app/routers/events.py`	Event creation, invoices, payment status reconciliation
Staff & Sub-Teams	`app/routers/staff.py`	Event staff, wages, sub-contractors, payouts

Componentize Massive Frontend Modals

MAINTAINABILITY

`frontend/app/expenses/page.tsx` (4,300 lines) contains all settlement, edit, and print modals inline. Moving these into `frontend/app/expenses/components/` makes each modal self-contained and easy to update without risking regressions.

🚀 STEP-BY-STEP PRIORITY ROADMAP

PRIORITY	PHASE	SPECIFIC ACTIONS	IMPACT
PRIORITY 1	**Quick Cleanup**	Delete `services` phantom file, remove orphaned `events/edit` folder, remove duplicate schema in `schemas.py`, remove deprecated `eslint` from `next.config.ts`.	Removes 100% of dead code, zero build warnings, clean workspace.
PRIORITY 2	**Security Hardening**	Implement `passlib[bcrypt]` password hashing, issue signed JWT tokens for API authorization, and sanitize `/auth/roles`.	Prevents credential leaks and unauthorized role escalation.
PRIORITY 3	**Permanent DB Backups**	Generate complete Alembic sync migration and establish a 1-click scheduled local PostgreSQL `.bat` backup to your permanent drive.	Guarantees zero data loss and automated disaster recovery.
PRIORITY 4	**Architecture Refactoring**	Split `main.py` into 5 APIRouters and extract modal components from large frontend pages.	High modularity, faster updates, clean enterprise codebase.